

Manuale d'uso — correlazioni dinamiche trimero a catena aperta versione a stato esatto (`prepare_state`)

Note di lavoro — Tesi triennale, Samuele Schiavi
Relatore: Prof. Alessandro Chiesa

Scopo

Questo manuale spiega come eseguire la simulazione delle correlazioni dinamiche $C_{ij}^{\alpha\beta}(t)$ per il trimero a catena aperta nella sua versione *originale*: lo stato fondamentale $|\psi_0\rangle$ è preparato sul registro con le ampiezze esatte (`QuantumCircuit.prepare_state`), non con un ansatz VQE. È la versione da usare quando interessa isolare l'errore di Trotter e quello statistico (shot), senza mescolarli con un residuo di fedeltà dell'ansatz variazionale. Per la versione con il circuito VQE reale (W-2qC.K2), vedi il manuale gemello `manuale_uso_correlazioni_trimero_catena_vqe.tex`.

Prerequisiti

File necessari nella stessa cartella (tutti nella fase *correlazioni dinamiche*, trimero catena):

- `trimer_chain_exact.py` — Hamiltoniana `trimer_hamiltonian_dm`.
- `trotter_trimero_catena.py` — evoluzione $U(t) = e^{-iHt}$ via Suzuki-Trotter (due bond, opzione `alternate` disponibile, default `False`).
- `circuito_correlazioni_trimero_catena.py` — il circuito Hadamard test a 4 qubit.
- `validate_circuito_correlazioni_catena.py` — riferimento classico esatto (`scipy.linalg.expm`).
- `validate_shot_noise_trimero_catena.py` — validazione a shot finiti.
- `scan81_trimero_catena.py` — scan completo delle 81 combinazioni.
- `generate_figure_shotnoise_catena.py`, `generate_figure_scan81_catena.py` — figure.

Librerie Python: `qiskit` (2.x), `qiskit-aer`, `numpy`, `scipy`, `matplotlib` (solo per le figure).

Importante, a differenza dell'anello: questa fase lavora su **due** punti di lavoro, non uno solo, tenuti entrambi per confronto perché non ancora unificati con il relatore (vedi `simmetrie_correlatori_trimero_catena.tex`, nota aperta):

- **VQE-DM:** $J = 1$, $b = b_c = 3.0$, $D = 0.15$ — punto dove è stato identificato l'ansatz canonico W-2qC.K2.
- **S0 (Trotter):** $J = 1$, $b = 3.0073414$, $D = 0.3$ — punto provvisorio della fase Trotter, verificato essere l'argmin del gap a $D = 0.3$.

Gli script che supportano entrambi i punti li selezionano con un argomento da riga di comando (`vqedm` o `s0`, default `vqedm` se omesso); altri (i moduli di validazione) eseguono automaticamente entrambi in sequenza. $t = 1.3$, $N = 200$ passi di Trotter in tutta la fase, in entrambi i punti.

Passo 1 — un correlatore singolo, da riga di comando Python

Per calcolare un correlatore specifico, ad esempio $C_{11}^{yy}(t=1.3)$ al punto VQE-DM:

```
from circuito_correlazioni_trimero_catena import (
    ground_state, correlator_from_circuit,
)

J, b, D = 1.0, 3.0, 0.15
psi0, E = ground_state(J, b, D)

C = correlator_from_circuit(1, 'y', 1, 'y', t=1.3, N=200,
                           J=J, b=b, D=D, psi0=psi0)
print(C)    # atteso: un numero complesso, |C| dell'ordine di alcuni decimi
```

Non passare `ansatz_params`: di default resta `None` e la preparazione avviene con `prepare_state(psi0)`, ampiezze esatte. Per il punto S0, usare invece $b=3.0073414$, $D=0.3$.

Passo 2 — validazione contro il riferimento classico esatto

```
python validate_circuito_correlazioni_catena.py
```

Esegue **automaticamente entrambi** i punti di lavoro in sequenza (nessun argomento da riga di comando). Per ciascuno stampa: confronto su cinque correlatori rappresentativi e tre valori di t (residuo atteso $\sim 10^{-3}$, errore di Trotter puro); zeri predetti a $t = 0$ (Cor. zero-t0-same/diff, atteso 0 a precisione macchina); relazione P_{13} ($C_{11}^{xx} = C_{33}^{xx}$, entro l'errore di Trotter); convergenza in N (rapporto ≈ 2 raddoppiando N).

Passo 3 — validazione a shot finiti

```
python validate_shot_noise_trimero_catena.py vqedm
python validate_shot_noise_trimero_catena.py s0
```

A differenza del Passo 2, questo script richiede l'argomento (`vqedm` o `s0`): esegue un solo punto per lancio, non entrambi. Separa i due contributi di errore (Trotter e statistico) su $C_{11}^{zz}(t=1.3)$: rapporto statistico/Trotter $\sim 10\times$ a VQE-DM, $\sim 115\times$ a S0 (vedi `circuito_correlazioni_trimero_catena_spiegato.tex`). Salva `shot_noise_catena_{vqedm,S0}_results.npz`.

Passo 4 — scan completo delle 81 combinazioni

```
python scan81_trimero_catena.py vqedm
python scan81_trimero_catena.py s0
```

Come il Passo 3, un punto per lancio. Richiede qualche minuto per punto (81 combinazioni $\times 2$ parti Re/Im $\times$ (statevector + 8192 shots), con cache di trascompilazione). Stampa un riepilogo (nessuno zero strutturale confermato per la catena, errori medi/massimi, correlatori più "ricchi") e salva, per ciascun punto:

- `scan81_catena_{vqedm,S0}_results.npz` — matrici 9×9 di riferimento classico, statevector e shot, più le statistiche di errore.
- `scan81_catena_{vqedm,S0}_results.json` — stessi dati, riga per riga.

Attenzione: percorsi relativi — va eseguito da dentro la cartella di lavoro.

Passo 5 — figure

```
python generate_figure_shotnoise_catena.py
python generate_figure_scan81_catena.py
python generate_circuit_fig_trimer_catena.py
python generate_figures_circuito_trimer_catena.py
```

I primi due richiedono che i file `.npz` dei Passi 3–4 (**entrambi** i punti) siano già presenti: generano automaticamente una figura per punto. Gli ultimi due generano il diagramma del circuito e le tre figure di `circuito_correlazioni_trimer_catena_spiegato.tex` (convergenza Trotter, validazione continua, contrasto "nessuno zero sul sito 2") – questi ultimi lavorano solo al punto VQE-DM, hard-coded in testa allo script.

Problemi comuni

- `ModuleNotFoundError` su uno dei moduli locali — verificare di eseguire dalla cartella che contiene tutti i file elencati in "Prerequisiti".
- `FileNotFoundError` su uno dei file `scan81_catena*_results.npz` — va prima generato eseguendo `scan81_trimer_catena.py` col punto corrispondente (Passo 4); non incluso di default perché il calcolo richiede qualche minuto per punto.
- Risultati leggermente diversi da quelli riportati nei documenti di teoria — controllare il punto di lavoro (J, b, D) : due punti diversi sono in uso in questa fase (vedi riquadro sopra), a differenza dell'anello che ne ha uno solo. Controllare anche t, N : hard-coded in testa a ogni script.
- Confusione fra le due topologie — questo modulo importa da `trimer_chain_exact.py` e `trotter_trimer_catena.py`, **non** dagli omonimi `_ring_/_anello_`: le firme delle funzioni differiscono (J, b, D qui, J, Jp, b, D nell'anello — nessun parametro Jp nella catena).

Riferimenti

Teoria e derivazioni: `simmetrie_correlatori_trimer_catena.tex`, `circuito_correlazioni_trimer_catena_spiegato.tex`. Catalogo completo della fase: `trimer_catena_correlazioni.tex`.